```
  def self.num_fulfilled=(new_value)
    @num_fulfilled = new_value
  end

  def self.num_fulfilled
    @num_fulfilled || 0
  end

end
```

Because threads share memory, if you have five Sidekiq jobs each running in a thread and all five call `Fulfiller.new.start_fulfillment(order)`, all five will attempt to modify the class variable `@num_fulfilled`. Depending on how the underlying virtual machine instructions are ordered, the value of `@num_fulfilled` will be unclear and possibly corrupted.

Concurrency issues can also happen when external resources, like your database, are shared. If you recall `AddMoreProductsJob` from *Understanding Idempotence*, it updates a row in the database. If two of that job were queued for the same product but with different quantities, it's impossible to predict what quantity would eventually be stored in the database.

Avoiding Problems with Concurrent Execution

Like idempotency, it's difficult to analyze code to determine that it's safe to be executed concurrently with other code. It requires you to play out the same code happening at the same time in your head and decide if there is a case where the ordering of operations creates a problem. And, since any two blocks of code could potentially be executed concurrently, it's almost impossible to itemize every possible behavior.

Here are a few tips to help avoid the most common issues:

1. Avoid the use of `mattr_accessor` or `catt_accessor`. These create a shared global state, which you almost never need. They are worse than that, however, because they allow that state to be manipulated by instance variables. This allows you to write concurrency-unsafe code that appears to be safe. There is no need to use these methods.